

OkBuddy CV Workflow - Product Specs

UX Principles

- One goal at a screen. One step at a time
- Gamified progress (via CV Score).
- Free vs. premium UX separation.
- Maximum simplicity and friction reduction. Simplify, simplify, simplify. Every detail, button, text, interaction ... must be dead simple, intuitive, frictionless
- Design with monetization in mind
 - We try to design & offer the best user experience, then find smart ways to monetize our features
- Visual & Interaction Guidelines:
 - Clean and professional: modern, minimalist interface aligning with industry standards.
 - Consistent colors and fonts: primary colors soft cyan (#E0F7FA), gentle mint green (#E6F4EA), light sky blue (#E3F2FD); typography professional yet friendly.
 - Intuitive interactions: clear affordances, intuitive buttons, and CTAs; maintain consistency across interactions.

High-level USP

Step	Problem/ Need	OkBuddy USP
Updating Personal Information & Recent Experiences	As a job seeker, I want to efficiently update my CV with my latest experiences and achievements, so that my CV always reflects my most relevant and recent qualifications. Currently, I have to manually edit and decide what details to include, which is tedious, confusing, and often results in missing or poorly represented information.	Guided conversational AI prompts to efficiently capture essential updates (new roles, achievements, skills). Automatic AI suggestions on relevant experiences or certifications users might overlook based on industry and job role trends. Intelligent phrasing and formatting recommendations provided instantly by the LLM. OkBuddy 10x Improvement: Transforming manual, unclear, and error-prone updates into an

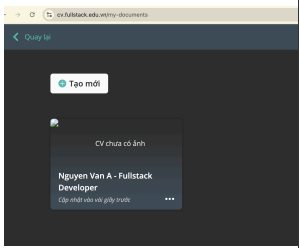
		efficient, guided, and precise process.
Showcasing Experiences with Strong, Impactful Language	As a job seeker, I want my experiences and achievements to be articulated powerfully and professionally, so that I can impress hiring managers and stand out from other candidates. Currently, I have to write descriptions manually and struggle with effectively articulating my achievements, making the process frustrating and often resulting in generic, weak descriptions.	LLM-Powered Solution: AI-driven experience rephrasing tool that transforms simple inputs into compelling, achievement-oriented descriptions. Automatic extraction and suggestion of quantifiable metrics and impactful language tailored specifically to industry and role standards. Instant variations to ensure personalized and authentic representations of user experiences. OkBuddy 10x Improvement: Professional-grade, impactful resume content generated in seconds, overcoming manual writing weaknesses.
Tailoring CV to Specific Job Requirements	As a job seeker, I want to easily tailor my CV specifically to each job's requirements, optimizing it for ATS systems and recruiters alike, so that my resume consistently catches the attention of hiring managers and gets past automated filters. Currently, I have to manually guess job-specific requirements and ATS screening criteria, repeatedly adjusting my CV through trial-and-error, which is tedious, anxiety-inducing, and risks immediate rejection or	LLM-Powered Solution: Real-time AI-driven analysis of job postings to instantly identify critical keywords and role-specific requirements. Automated tailoring of CV content, restructuring and rephrasing skills and experiences to perfectly match the specific job description. ATS optimization checker integrated to proactively ensure formatting and keywords are optimized for both automated systems and

	oversight by recruiters who spend only seconds scanning each resume.	recruiters' rapid initial scans. OkBuddy 10x Improvement: Dramatically reduces customization time from hours to seconds while significantly boosting ATS pass rates and recruiter attention.
Reviewing & Finalizing CV	As a job seeker, I want to confidently review and finalize my CV knowing it meets high professional standards, so that I can submit applications without hesitation or anxiety. Currently, I rely on self-review or uncertain methods to check my CV's quality, leading to anxiety about unnoticed errors or missed opportunities to make a strong impression during the critical initial recruiter scan.	LLM-Powered Solution: AI-powered final review providing immediate, actionable insights to enhance CV quality. Instant scoring of CV effectiveness, clarity, and alignment with professional standards and job-specific requirements. Real-time improvement recommendations ensuring high-impact, error-free submissions. OkBuddy 10x Improvement: Removes guesswork and anxiety from final CV reviews, ensuring confident submissions that clearly stand out.
Download updated CV		

Requirements

Step 0: Workspace & Navigation

Problem/ Need	Solution/ Feature	Pain Severity/ Impact	Status
As a returning user, I want to manage multiple customized	Workspace/ CV Management	Critical	Cursor Ready

CV versions for different job applications So that I can easily keep track and efficiently apply to multiple roles. Currently, I struggle to remember which CV matches each specific job, causing confusion and inefficiency.			
	Clear, visual thumbnails or summary views of each CV.	High	Cursor Ready
	When receiving job post info, auto fetch company name (provided from job description) & label the current CV project users are working on	High	Cursor Ready
	Easy search/filter by job title or date.	Low	Cursor Ready
As a user, I need to clearly understand my progress through the CV creation process So that I remain motivated and aware of what steps remain. Currently, it is unclear how many steps there are and my exact position within the process, which causes uncertainty and frustration. — As a job seeker, I want a focused CV editing workspace with minimal distractions So that I can concentrate on one	Clean, minimal interface with only essential elements visible (e.g. a progress indicator and the current step's instructions) to uphold one clear goal per screen.	Critical	Cursor Ready
	A progress tracker (such as a progress bar or step list) that updates as the user advances, reinforcing a sense of progress and providing context about where they are in the journey.	High	Cursor Ready
	Linear step-by-step navigation controls (a prominent “Next” button and a disabled or subtle back button	High	Cursor Ready

task at a time without feeling overwhelmed by too many options.	when not applicable) ensuring one primary action per step and preventing user confusion.		
	Intuitive microcopy and tooltips on navigation elements (e.g. labels or icons for steps) so the user immediately understands what to do next with zero friction.	High	Cursor Ready
	Return users back to Workspace (CVs Management) screen when they click Exit (“Thoát”) or drop off, close the browser ... anytime during the CV preparation workflow.	High	Cursor Ready
As a user, I want to know my work is safe and be able to resume easily, So that I feel confident exploring the app without losing progress.	Continuous autosave functionality from the start of the workflow, preserving every edit or upload in real time. This allows users to navigate away or refresh without losing any changes (no manual save needed). Users can go back or forward any step without losing any data from any step	Critical	Cursor Ready
	Persistent workspace state — when returning to the website/ app, users find themselves at the same step with their data intact (e.g. uploaded CV and	High	Cursor Ready

	inputs retained), supporting a zero-friction stop-and-resume experience.		
	A minimal header with access to core actions that does not distract from the main flow, ensuring the user feels in control but never pulled away from the primary task: Top left: OkBuddy logo Top right: The first character of the user name (pulled from registration name or email)	High	Cursor Ready
As a user, I want to be able to download my CV at any step whenever I find it's good enough	Download button		Cursor Ready
As a user, I need to be able to continuously work on my CV according/ comparing to OkBuddy suggestions	OkBuddy recommendations sticky on the left hand side		Figma Needed
	Users' CV sticky on the right hand side reflect real-time updates/ improvements		
	Users CV's OkBuddy evaluation score sticky, clearly visible on screen This score will disappear after users make any change to		

	their CV (if we let it on all the time, whenever users make a change to their CV, the score gets updated, which could lead to server/ bandwidth issue) This score section has a button “Evaluate” to click on whenever users need to have		

Step 1: Upload Existing CV

User Action

On the first step, the user is prompted to upload their CV (e.g. drag-and-drop or file picker). An optional text field invites them to paste a job description, but it’s clearly marked as skippable (e.g. a “Skip for now” link). There are no lengthy forms or required profile setups – just this one primary step. By limiting upfront input to the resume (and an optional JD), we minimize cognitive load and get the user started immediately [OO]. Crucially, the user isn’t forced to sign up or fill every detail at this stage; allowing users to skip non-essential steps ensures we don’t turn off those who want to jump in quickly [OO]. This “value-first” approach (no account creation or multi-step tutorial required) respects the user’s time and builds trust by delivering value before asking for anything in return [OO].

Rationale

The goal is to maximize time-to-value – the quicker the user sees something useful, the better. By letting them start with just a CV upload, we remove barriers to entry (no overwhelming form asking for every CV section). If they have a target job, they can input it for personalized analysis, but if not, they can still proceed. This approach follows best practices of onboarding: keep it short, optionalize the extras, and let experienced or eager users skip ahead. The single prominent action (uploading a CV) on this first screen adheres to the “one action per screen” design principle, keeping the interface focused and intuitive

Problem/ Need	Solution/ Feature	Pain Severity/ Impact	Status
---------------	-------------------	-----------------------	--------

	Clean, minimal interface with only essential elements visible (e.g. a progress indicator and the current step's instructions) to uphold one clear goal per screen. A progress tracker (such as a progress bar or step list) that updates as the user advances, reinforcing a sense of progress and providing context about where they are in the journey. Linear step-by-step navigation controls (a prominent "Next" button and a disabled or subtle back button when not applicable) ensuring one primary action per step and preventing user confusion. Intuitive microcopy and tooltips on		
--	--	--	--

	navigation elements (e.g. labels or icons for steps) so the user immediately understands what to do next with zero friction. Return users back to Workspace (CVs Management) screen when they click Exit (“Thoát”) or drop off, close the browser ... anytime during the CV preparation workflow.		
As a first-time user, I need to quickly and effortlessly upload my existing CV so that I can start customizing it immediately.	Drag-and-drop CV upload area and a file selector button that accept common formats (PDF, .docx, etc.), minimizing friction in getting started. The upload field is centrally located and clearly labeled “Upload Your CV” – one clear action on this screen.		
	Guidelines	Medium - High	TBD

	Immediate parsing of the uploaded CV's content in the background, with a quick visual confirmation (e.g. file name shown or a checkmark) once the file is uploaded. This assures the user the upload was successful and the system is working on it.		
	Validation and feedback on upload – if the file is incompatible or empty, the user sees an intuitive error message guiding them to fix it (ensuring zero frustration if something goes wrong).		
As a first-time user, I want to create new CV from scratch	Default CV templates	Low	
I only have LinkedIn profile		Low	
As a job seeker who has a specific job in mind, I want to provide the job description along with my CV So that the tool can tailor its feedback to that role. Currently, I have to manually compare my CV to a job post, which is tedious and easy to miss	An optional field to add a Job Description (either paste text or upload a JD file) on the same screen as the CV upload. It's clearly labeled "Add a Job Description		

details. — As a job seeker, I need to provide a job description to precisely tailor my CV So that my qualifications align closely with the job requirements. Currently, manually comparing my CV to each job description is tedious and prone to oversight.	(optional)” with a short note like “for tailored suggestions,” so users understand its purpose.		
	Text limiter (for manual JD pasting)		
	JD File converter (Supported format: .pdf OR .docx)		
	If a JD is provided, a quick confirmation or preview (like “Job Description added: Product Manager at XYZ Corp”) appears, assuring the user that the information is captured. The UI then highlights the next action (e.g. a “Analyze CV” or auto-advance) to maintain a single primary focus.		
As a user, I want to skip the JD providing part when I only want to work on a generic CV yet So that I can jump straight on working on my CV	Skip logic that allows the JD field to be left blank – if no job description is provided, the app seamlessly proceeds with a		

	generic CV analysis. There's no intrusive prompt forcing a choice; the user can simply Next past it, aligning with zero-friction design.		
As a cautious user, I want to be assured my personal data is safe when I upload my CV, So that I feel comfortable using OkBuddy. (Uploading a resume can feel risky without trust.)	A brief note on the upload screen about privacy (e.g. "Your data is secure and used only for your CV analysis") or an icon linking to our Terms of Service/ Privacy, building user trust at the very first interaction.		
	Clear confirmation and transition after uploading (such as a prominent "Continue" button once the CV is uploaded, or an automatic advance after a short countdown), which gives users a sense of closure for this step and gently moves them to the next.		

--	--	--	--

Step 2: CV Analysis & Evaluation

System Response

Immediately after uploading (and providing a JD, if they did), OkBuddy generates an **instant CV analysis**. If a job description was provided, the user is shown a “**Match Score**” – e.g. “*Your CV matches 68% of this job posting.*” This score is displayed prominently (often as a percentage or gauge) along with a brief highlight of key findings. For example, the interface might say: “*Match Score: 68 – Your resume aligns fairly well. You match 12 of 20 required skills, but you’re missing 3 of the top keywords.*” The user sees this result *right away*, possibly on a dedicated results screen or modal, as soon as the scan is complete. (If no JD was provided, this step can instead show a generic **CV Strength indicator** or a message like “CV uploaded! Let’s improve it,” then lead straight into editing mode – ensuring even users who skipped the JD get immediate feedback or at least a confirmation that their resume is ready for editing.)

Design Details

The **match score** is derived from comparing the CV content to the job description text. This concept is similar to what advanced resume tools do – for instance, Jobscan gives “*an instant analysis of how well your resume is tailored for a particular job*” and calculates a match rate based on resume vs. JD alignment . Presenting such a score up front serves as an “aha!” moment for the user, showing value in seconds. It’s the same strategy behind tools like website graders that give you an immediate score from a single input, hooking the user quickly . Alongside the score, we include a few key insights in plain language (to avoid overwhelming the user with too much data). For example: “*Your resume is ATS-friendly*  ” or “*3 important skills from the job description are missing from your CV.*” This instant feedback **captures the user’s attention** and provides a starting point for improvement.

Rationale

Showing a result immediately after upload delivers **fast value** when user attention is highest. It reduces the time to first benefit, which keeps users engaged and curious to continue . The match score in particular personalizes the experience – the user sees a metric that matters to their goal (getting a job) without having to do anything else. This early win encourages them to proceed to the next steps. By making the job description **optional**, we ensure that users without a specific job in mind still move forward (they aren’t forced to find a posting just to use the tool). In either case, the user gets feedback right away: either a match score (with a JD) or a general CV assessment (if no JD), reinforcing that *something valuable has already been gained*. This balances **early engagement over full data completeness** – we prefer to give a quick, motivating output from partial data rather than demand complete information upfront. (Notably,

tools like Zety also found success by giving users a resume “score” and tips once their resume is input – it’s a proven way to motivate improvement.)

Problem/ Need	Solution/ Feature	Pain Severity/ Impact	Status
As a job seeker, I want immediate feedback on my CV in the form of a score or match percentage, So that I instantly know how well my CV is doing and feel motivated to improve. Currently, without any feedback, I’m left guessing if my resume is good enough.	A prominently displayed CV Score or match percentage right after upload. For example, “CV Match Score: 65/100” (if a JD was provided, this reflects how well the CV matches the job; if not, it reflects overall CV strength).		
	Animated score reveal for gamified feedback – the score might count up or a gauge graphic might fill to the score level, creating a moment of reward/anticipation as soon as the analysis is done.		
	A subheading or color-coded assessment that explains the score in plain language (e.g. “Moderate match – Your CV overlaps some with the		

	job needs, but there's room to improve" or "B Grade – Good foundation, but could be stronger"). This intuitive copy ensures the user understands what the number means at a glance.		
	Visual indicators accompanying the score, such as a progress bar or ring that is 65% filled, to reinforce how close the CV is to an optimal state. This ties into gamification by implicitly showing the user "how far you've come, how far to go."		
	One clear call-to-action on this screen, such as a "Improve My CV" button or a down-arrow icon to scroll into suggestions. The interface at this stage is minimal: just the score, a brief message, and the next action, encouraging the		

	user to continue without distraction.		
As a user, I want a hint of what needs improvement along with the score, so that I have context for the number and know that the tool will help me fix things.	A concise summary of key findings next to the score (e.g. “3 critical issues found”) or an indication like a checklist teaser (“Strengths: Formatting; Needs work: Keywords, Achievements”). This remains high-level to avoid overload, but it piques interest in the next steps.		
	A “View Detailed Report” link or button near the score that is marked with a lock icon (Pro feature). If clicked, it prompts the user to upgrade to see an in-depth breakdown (e.g. category-by-category scoring, ATS compatibility, etc.), thus introducing premium value without hindering the free flow.		
	Instant		

	feedback speed – the analysis occurs within seconds, and if it takes a moment, a simple loading animation (“Analyzing your CV...”) keeps the user informed. This quick turnaround reinforces a seamless, responsive experience.		

Step 3: Branching Paths – Generic vs. Tailored Optimization

Based on the user’s input in Step 1, the flow now branches to provide the most relevant editing experience:

No Job Description (Generic CV Editing Mode)

The user enters a general resume editing interface focused on overall **CV improvement**. Here, OkBuddy highlights universal enhancements that can make the CV stronger. For example, it might flag sections that are blank or weak, identify bullets that could be more impactful, or point out formatting/ATS issues (like too many graphics or improper headings). The UI could present a simple checklist or “to-do” list for the resume: e.g. *“Add a profile summary,” “Fix inconsistent date formats,” “Use more action verbs in Experience.”* Each suggestion is actionable. The tone is encouraging, not critical. The user can click on a suggestion to jump to that section of their CV or see tips for how to improve it. **Wireframe logic:** This could be a two-pane layout – the left side listing improvement suggestions or sections (with indicators), and the right side showing the resume content ready for editing. The user can freely edit text inline or accept automated suggestions (some of which may be premium – more on that shortly). The key is that, without a target job, the focus is on **general best practices** and making the resume shine broadly (e.g. ensuring clarity, good structure, quantifying achievements, ATS compliance, etc.). The system might even compute a **Resume Quality Score** out of 100 that updates as they make changes, to gamify the process.

With Job Description (Tailored CV Optimization Mode)

If the user provided a JD, the editing interface adapts to focus on **tailoring the CV to that job**. The user likely sees the job description or its key requirements summarized in the UI. For instance, a side panel might list the top skills and keywords from the JD and indicate whether they appear in the resume or not. (This mirrors how some tools operate: e.g. the Jobscan report shows a list of hard skills from the job description and how many times each appears in the resume .) OkBuddy will highlight missing keywords or experience areas – e.g. *“The job mentions **Project Management** 5 times, but your CV doesn’t. Consider adding a relevant project or skill.”* It might also flag content in the CV that could be phrased more like the JD (for instance, if the JD says “customer support” and the CV says “client service,” it could suggest aligning terminology). Essentially, the user is guided to bridge gaps between their resume and the target role. **Wireframe logic:** Similar two-pane layout, but the left panel is now contextual to the job: maybe divided into sections like “Missing Skills,” “Keyword Matches,” “Job Requirements Coverage,” etc., each with specific suggestions. The right pane still shows the resume text. The JD text might be accessible via a toggle or modal if needed for reference. The match score from Step 2 remains visible as a gauge or number here, providing motivation – as the user optimizes their resume, they can see this score potentially improve in real-time. This creates a game-like feedback loop (e.g. *“Match 68% -> 85%”* after changes).

Rationale

Branching the flow ensures the user’s experience is **relevant to their immediate goal**. If they’re just looking for general CV improvements, we don’t bog them down with job-specific data (which could confuse or overwhelm someone who isn’t targeting a job yet). Conversely, if they do have a job in mind, we make every suggestion count toward landing that job. This conditional routing embodies a personalized UX – it “routes them into tailored optimization” only when applicable, providing a more frictionless feel than a one-size-fits-all workflow. It’s essentially contextual onboarding: we *either* guide the user through generic improvements or a targeted tailoring process. In both cases, the interface is minimal and action-focused. The user isn’t presented with a massive form or multiple inputs at once; instead, they see their resume and a **focused list of next-step actions** relevant to their scenario. This approach keeps cognitive load low because the user only deals with relevant suggestions, one context at a time. It also provides **instant relevance**: a user who came with a specific job goal in mind sees that the system is addressing their need (tailoring to the job), while a user who just wanted a quick CV touch-up isn’t forced to think about a job posting. Overall, branching increases user satisfaction by *meeting them where they are* in their journey.

Problem/ Need	Solution/ Feature	Pain Severity/ Impact	Status
As a job seeker targeting a specific job,	Automatic Tailored Mode		

I want the CV improvement process to tailor suggestions to the job description I provided, so that the changes I make will directly increase my fit for that role. Currently, I'd have to manually align my CV with the job post, which is error-prone and slow.	activation when a JD is provided. The system recognizes the JD and adjusts all upcoming suggestions to focus on matching the CV to that job (e.g. prioritizing missing keywords or relevant experience). The UI might display a label like "Tailored to: [Job Title]" to remind the user that the advice is job-specific.		
	Side-by-side comparison availability in tailored mode – for instance, a toggle or modal where the user can quickly view the JD content next to their CV. This gives context for the suggestions (like highlighting a requirement from the JD that's not yet reflected in the CV) without cluttering the main flow.		
	Contextual guidance messages that set the stage:		

	e.g. after the score, a short note saying “Now let’s improve your CV for the [Job Title] role.” This confirms the tailored path and makes the user feel the app understands their goal.		
As a user without a specific job target, I want the improvement suggestions to be generic best practices, so that my CV becomes stronger overall for any application. I’m not focusing on one role right now, so I need broad improvements.	Automatic Generic Mode when no JD is given. The suggestions presented will be based on general CV optimization (structure, clarity, impact), and the UI might simply say “General CV Improvements” to indicate the scope.		
	Industry/role selection prompt (optional, free) – if no JD is provided, the tool might ask “Which industry or role are you targeting?” in a non-mandatory way to slightly customize generic advice (for instance, picking		

	“Marketing” could tailor tone or examples). This is offered as a one-click dropdown, keeping the flow quick and optional.		
	Confirmation message for generic path: e.g. “No specific job chosen – we’ll apply general best practices to polish your CV.” This assures the user they’ll still get valuable feedback even without a JD, maintaining engagement.		
As a user, I want the flexibility to switch or add a job target if my situation changes, so that I can get tailored advice without restarting the process.	Change/Add Job option available during the editing process for Pro users – for example, a button “Tailor to a different job” that lets the user input a new JD or swap the existing one. The system would then update the suggestions list for the new target. This caters to users applying to multiple jobs (a premium use		

	case) and reinforces Pro value.		
	Seamless branching logic – users who skip adding a JD simply continue to the next step without any extra clicks (no “choose path” friction). Users who did add a JD are automatically funneled into tailored suggestions. The app doesn’t present an unnecessary fork for the user to think about; it “just works” based on their input.		

Step 4: Guided Editing – One-Step-at-a-Time Improvements

Now the user begins improving their resume using the OkBuddy editor. The design here remains **minimal and guided**, ensuring the user focuses on one fix at a time rather than feeling lost in a complex tool.

Interactive Editing Flow

The editing interface is interactive and tutorial-like. Rather than simply dropping the user into a raw text editor with a bunch of advice, OkBuddy uses a **stepwise guidance** approach. For example, the top of the suggestion panel might say “**Step 1 of 5: Improve your Summary**” with a short tip (if in generic mode) or a job-tailoring hint (if in tailored mode). The user can click a button like “Auto-Improve” or “Show Suggestion” to get an AI-generated improvement for that section (this is one of the premium features – see monetization below), or they can manually edit it with the tip in mind. Once they address that item (or choose to skip it), the interface

advances to **Step 2: e.g. “Add a measurable achievement to your Experience section”**, and so on. In this way, *each step of the editing flow involves a single clear focal action*: e.g., fix this bullet, tweak this section title, add a keyword here. The UI might visually emphasize the current suggestion (highlighting the relevant part of the resume text and perhaps greying out the rest). Navigation is simple: a “Next suggestion” arrow or button moves to the next tip, and users can always go back or view a list of all suggestions if they prefer a non-linear approach. Importantly, there’s always just **one primary call-to-action on screen at a time** (like “Apply this change” or “Next”), which keeps the user from getting distracted or overwhelmed. This adheres to the one-action-per-screen principle common in good UX design – by guiding users through a series of small, digestible tasks, we prevent decision paralysis and make the process feel easy.

Visual Feedback

As the user makes changes, we provide positive feedback. For instance, a small animation might show their overall CV score ticking up, or a checkmark appears next to the completed suggestion in the list. If in tailored mode, the match percentage might increase or a missing skill count might decrement (e.g. *“Missing skills: 3 → 2”* after they add one). This real-time feedback keeps users motivated to continue. It also reinforces the value of each action they take – each edit clearly makes their CV “better” in the eyes of the system (and presumably employers/ATS). By chunking the editing into these micro-tasks, we reduce cognitive overload; users tackle improvements one by one, which is a classic **progressive disclosure** technique to handle complex tasks . At any point, the user can also freely scroll and edit the resume outside the guided flow (for power users who know what they want to change), but the guided path is always there to lead them through all key optimizations.

Rationale

This guided, stepwise editing flow is designed to keep the UI **action-focused and uncluttered**. Users never face a blank slate or a daunting list of “everything wrong with your CV” without direction. Instead, they are coached through improvements in a logical order. This dramatically lowers the cognitive effort required – the user doesn’t have to figure out *where to start* or *what to do next*, the system gently tells them. Short, focused steps also align with how users process information; by **breaking down the editing into bite-sized actions, each with a clear goal**, we make the overall task (improving a resume) much less intimidating. This approach is in line with UX best practices for avoiding overload: guiding users with familiar patterns (like a stepper), providing feedback, and simplifying choices at each moment . By the end of this guided flow, the user will have iteratively improved multiple aspects of their CV without ever feeling lost. They remain engaged because each step is achievable and yields an immediate reward (e.g. a better score or a visual checkmark). In essence, Step 4 ensures the user can painlessly act on the insights from Steps 2 and 3, keeping them motivated and on track.

Problem/ Need	Solution/ Feature	Pain Severity/ Impact	Status
---------------	-------------------	-----------------------	--------

As a job seeker, I want to address CV improvements one step at a time, so that the process feels manageable and I don't get overwhelmed by too many changes at once.	One-at-a-time suggestion flow – the interface presents a single improvement suggestion per screen (or card). Each suggestion comes with a short title/summary (e.g. "Add a Professional Summary") focusing the user on one clear goal. This guided approach aligns with the "one action per step" principle, simplifying decision-making		
	Simple navigation for suggestions: a prominent "Next" button to move to the next suggestion after completing the current one (or "Done" if it's the last), and a subtle "Back" if the user wants to revisit the previous tip. This linear flow keeps the user focused on the current task without distraction.		
As a user,	Inline issue		

I want the tool to clearly highlight what needs to change in my CV, so that I instantly know where to focus my editing without having to guess. Currently, if I get a generic suggestion, I'm not always sure which part of my resume it refers to.	highlighting on the CV preview/editor. For example, if the suggestion is "Add more role-specific keywords," the tool highlights sections of the CV lacking those keywords or perhaps highlights the keywords in the JD that are missing from the CV. This visual cue directs the user's attention precisely.		
	Contextual markers or annotations: small icons or underlines in the text where an improvement is recommended (e.g. an exclamation mark icon next to a job title that needs a date, or highlighted text that could be stronger). Clicking the marker can show more details or the suggestion text, integrating guidance directly on the CV.		

	For tailored mode, missing keywords from the JD are listed or highlighted clearly (e.g. “These keywords are not in your CV: Salesforce, Budget Management”) and the user can see where to incorporate them. For generic mode, common issues like “passive language” might highlight phrases to rewrite. The UI remains minimal by showing highlights only for the current suggestion to avoid clutter.		
As a user, I want to edit my CV directly in the app where I see the suggestion, so that I can immediately apply the advice without switching to another editor or breaking my flow.	Inline CV Editor embedded in the suggestion screen. The user’s CV is displayed in an editable format (preserving basic formatting like bullet points). When a suggestion is shown, the relevant section of the CV is either directly editable or an edit field is provided right		

	below the suggestion.		
	Click-to-edit interaction: e.g. if the suggestion says "Add a summary at the top," the user can click a placeholder at the top of the CV to start typing one. Or if the suggestion is "Improve this bullet," clicking the highlighted bullet opens a text box for editing just that item. This reduces friction by eliminating extra navigation or separate modals for editing.		
	Changes reflect immediately in the CV preview. As the user types or edits, they see the updated text in context (maintaining a WYSIWYG view of their resume). This real-time feedback ensures the user knows exactly how their CV is shaping up.		

As a user, I want my changes to save automatically as I make them, so that I never worry about losing progress or having to redo work.	Autosave in editor – every change the user makes (every keystroke or every few seconds of idle) triggers an automatic save to the server. A small “Saving...” then “Saved” indicator can reassure the user that their progress is secure without any manual action.		
	Robust undo/redo support (in-place) for the CV text edits, so users can experiment with changes risk-free. Even without a manual save, they can revert an edit if needed, which adds confidence to edit freely.		
	Draft retention: if the user temporarily leaves the editing (e.g. goes back to the dashboard or loses connection), the partial edits on the current		

	suggestion are not lost. When they return, the text appears as they left it, preserving their effort (sunk-cost reinforcement to continue editing).		
As a user, I want to see my CV score improve as I apply suggestions, so that I feel a sense of accomplishment and know the impact of each change.	Dynamic score update – after completing each suggestion (for instance, when the user clicks “Next” or marks a suggestion done), the CV Match Score recalculates. The new score is displayed prominently, replacing or updating the old score in the corner of the screen.		
	Score change animation: whenever the score updates, it visually highlights the improvement (e.g. the score number could flash or increment with a plus sign “+5”, or a small upward animation). This gamified feedback clearly shows the user		

	that their action had a positive effect, reinforcing the value of doing each step.		
	Milestone celebrations for significant score thresholds (for example, hitting a score of 80/100 might trigger a brief confetti burst or a congratulatory message like “Great work! Your CV is now much stronger”). These playful elements encourage the user to keep going to see what happens at 100%.		
As a user, I want to track how many suggestions I’ve completed and how many remain, so that I can gauge my progress and stay motivated to finish. Currently, not knowing the end in sight can be demotivating if the process feels open-ended.	Progress checklist or counter – the interface shows the number of improvements done out of the total identified (e.g. “Improvement 3 of 8”). This could be a small text or icon indicator on the suggestion screen or a progress bar that fills with each completed item.		

	A collapsible checklist of all suggestions is accessible (for example, a “View all suggestions” link that shows a list of upcoming and completed tasks). Completed suggestions are check-marked or greyed out, while pending ones are plain. By default this stays tucked away to keep the UI clean, but users who want an overview can peek at it.		
	Progress feedback at the end of each suggestion: after the user applies a suggestion, a brief message might say “Nice! 3 out of 8 improvements done.” This explicit acknowledgment of progress uses sunk cost effect — the user sees how far they’ve come and is encouraged to finish the rest.		

As a user, if a suggestion doesn't apply to me or I'm not ready to change that aspect, I want the ability to skip it and move on, so that I'm not forced to do something that doesn't fit my situation.	Skip option for suggestions – every suggestion screen includes a secondary action like “Skip this improvement” in a less prominent way. If clicked, the system records that suggestion as skipped and immediately navigates to the next one, maintaining momentum.		
	Non-penalizing skip logic: skipped items are not counted as failures; they might simply not contribute to the score improvement. The final CV score might not reach 100% if key suggestions are skipped, subtly encouraging users to address them later without outright forcing it.		
	Reminder at end for skipped items: if the user skips any suggestions, at the end of the flow the tool can		

	list “Skipped improvements” with an option to revisit them. This provides a second chance in case they skipped out of confusion or time constraints, aligning with zero-friction philosophy by never truly closing a door.		
As a user, I want each suggestion to be written in clear, helpful language (with examples if possible), so I understand exactly how to implement it. Currently, some resume tools give cryptic advice that leaves me unsure what to do.	Intuitive suggestion copy – each improvement tip is phrased in plain language and often includes a “why” or “how”. For example: “Your CV has no summary. Why? A brief summary helps recruiters quickly learn about you. How to fix: Add 2-3 sentences at the top highlighting your experience and skills.” This coaching tone educates the user as they edit.		
	Inline examples or templates: where relevant, the suggestion can show a mini		

	example. E.g. for “Add metrics to achievements,” a small italic text might show “e.g. Increased sales by 20% in Q4” as an example of a metric-driven bullet. This appears right below the suggestion text, guiding the user with zero extra effort.		
	(Pro) AI assist for suggestions – Pro users have the option to get an AI-generated suggestion or rewrite. For instance, next to a suggestion like “This bullet point could be stronger,” a Pro user might see a button “Generate Example” that, when clicked, auto-fills a rewritten version or idea for that bullet using AI. The user can accept or tweak this, significantly reducing the effort needed for improvement.		

	(Pro) Enhanced grammar and style checking for Pro: beyond the core suggestions, Pro mode might also flag finer points (passive voice, excessive jargon, inconsistent formatting) and offer one-click fixes or suggestions. These additional tips maintain the one-at-a-time format and only appear for Pro users, giving them a more thorough polishing toolset.		
--	--	--	--

Step 5: Final Review & Completed | Integrated Monetization – Freemium Upsell in Context

Throughout the above steps, **monetization is seamlessly woven into the user journey** as a freemium model. The design carefully differentiates which features are free and which are premium (paid), doing so in a transparent and user-friendly way:

Free Core Features

All of the essential workflow described in steps 1–4 can be experienced for free. This includes uploading a resume, receiving a basic match score and feedback, and even seeing what needs improvement. Free users can manually edit their CV and implement suggestions on their own. They might also get access to some automated help in a limited capacity (for example, perhaps one or two AI-suggested rewrites as a free trial, or basic grammar fixes powered by simpler algorithms). The free tier essentially ensures a user can go from a raw resume to an improved resume **without paying**, giving them genuine value upfront. This is important – we treat free users as valued customers in their own right, not as bait. (Indeed, many will never pay, but their

positive experience can lead to word-of-mouth growth or future upgrades, so it's critical their journey is a good one .) For instance, free users might be able to improve their resume structure, see their match score, and get general tips at no cost. This approach is similar to other resume tools that allow basic usage for free – e.g. Resume Now lets you create and edit a resume without payment – while naturally reserving the most advanced functions for paid plans.

Premium (Paid) Features

Advanced **LLM-powered enhancements** and job-targeting tools are reserved for **OkBuddy Pro** (the paid tier). These premium features are the “power ups” for users who want to optimize further. Specifically, features like **Job-Specific Tailoring suggestions**, **“Rewrite for Impact” AI edits**, and **ATS Keyword Boost** (automatically adding or recommending keywords to beat applicant tracking systems) are behind a clear paywall. In the UI, these might appear with a lock icon or a label (e.g. a button might say “Try **Pro** AI Rewrite”). The user can see that these options exist, and often we'll let them trigger it to learn more – for example, clicking a locked feature will prompt a friendly modal: *“Unlock AI-Powered Resume Optimization with OkBuddy Pro!”* and briefly explain the benefits. We make sure to frame these premium features as **helpful enhancements** rather than obstacles . For instance, the messaging might be: *“Pro Feature: Instantly rewrite this bullet point to better showcase your impact using our AI.”* with a clear call-to-action to upgrade, instead of a harsh “Upgrade required to use this.” By showcasing what the feature **does** (the value it adds), we tap into the user's desire for the outcome, not just say “pay to continue.” Additionally, we might allow a **free preview or trial** of one premium action – e.g. let the user use the AI to rewrite one bullet for free, then show the result with a note “More AI improvements available in Pro.” This gives a taste of the value, increasing the chance the user sees the benefit and considers upgrading. The paywall is presented **transparently and at contextually relevant moments** – for example, right when the user is trying to do something that requires Pro (like generate a tailored cover letter or boost their match score from 85% to 95% with deeper AI tweaks). This timing is key: good freemium UX recognizes when a user is naturally ready to “level up” and offers the upgrade at that point, instead of random pop-ups . In OkBuddy, that means as soon as the user runs into the limits of free features (say they've fixed everything they can manually and their match score is still, e.g., 70%, the app might suggest: “Boost to 90% with Pro – add the missing keywords and polished phrasing in one click.”). All pricing info is communicated clearly and empathetically – no sudden surprises. A **Pricing & Plans** info link is always accessible (perhaps in the menu or on the upgrade modal), outlining exactly what extra features Pro gives (for example, *“Pro Features: Unlimited job scans, AI bullet point writer, ATS keyword booster, cover letter generator, etc.”*). This aligns with the guideline that monetization should feel like a natural part of the story, not an unrelated paywall that disrupts usage.

Example Monetization Touchpoints

To illustrate, imagine the user has uploaded their CV and a JD, gotten a 60% match score for free, and is now in tailored editing mode. They address a couple of free suggestions (like reordering sections or manually editing a line). Next, the system highlights a missing skill – *“The*

job requires Python – you haven’t listed it. Add a relevant experience or skill.” A “**Add with AI**” button might be next to this suggestion. If the user clicks it, a dialog could say: “🔥 *OkBuddy Pro can craft a custom bullet point showing your Python experience (using AI) and insert it into your CV. Upgrade now to use this feature.*” The value proposition is clear (get a custom-crafted addition to your resume) and it’s directly tied to the user’s current goal (adding Python to improve their match). This is far more compelling than a generic “Please upgrade for premium” message. It also doesn’t *force* them – they can still choose to cancel and maybe add something about Python themselves. But the seed is planted that Pro can save them effort and improve their outcome. Another touchpoint: at the end of the guided editing, after applying all free suggestions, we might show a summary: “*Great! You improved your CV. Match Score: 75%. With OkBuddy Pro, you could further increase your match and stand out even more – e.g. auto-optimizing phrasing and adding 3 high-impact keywords you’re missing. Want to see? [Upgrade to Pro]*”. By providing *specific* benefits and even quantifying them (like “3 high-impact keywords”), we frame premium as an upgrade in capability for the user, not a roadblock. This approach is recommended in freemium UX – highlight tangible benefits and even offer visual comparisons of what the user is missing by not upgrading . It’s a gentle nudge rather than a hard stop.

Rationale

The freemium design ensures that **free users get real value** (the app is useful even if you never pay), while making the **premium offering naturally appealing** once the user cares about fine-tuning their CV. This balances conversion goals with user experience. By integrating the upsell at relevant moments and communicating value clearly, we avoid user frustration. In fact, when done right, the user perceives the premium features as a helpful part of the journey (“I see why I’d want this”) rather than an arbitrary pay gate. This strategy echoes the philosophy that *the best UX monetization is to offer an experience worth paying for, yet still fair if not paid for* . Users who choose not to pay can still finish the process with a much-improved resume – they can download or export their CV (perhaps in a basic template) and their match score and improvements stay accessible. Users who do upgrade get the “wow” moments of one-click enhancements and a super-tailored CV, which justifies the cost. Importantly, **premium features are framed as empowering tools, not punishments** . Nowhere do we, for example, hide the match score behind a paywall or cripple the free resume with watermarks (common in some builders, but frustrating for users). Instead, we give free users a complete taste of success and then show how Pro can amplify that success. This approach fosters goodwill: even non-paying users feel respect and value, increasing the likelihood they’ll recommend OkBuddy or upgrade later when they need those advanced features.

Finally, the monetization is **transparent**: pricing is clear, and the upgrade process is simple (likely an in-app purchase or subscription that unlocks Pro instantly, so the user can continue their flow without interruption). By making monetization a seamless part of the UX (just another step when the user is ready), we avoid derailing the user’s journey. In summary, the freemium model is executed such that it **complements the user experience** – free users achieve a strong baseline CV, and power users can pay to turbocharge their CV for even better results.

This not only drives revenue but does so in a user-centric way, maintaining OkBuddy's intuitive and frictionless ethos even when asking for payment.

[illegible]

--	--	--	--

Through this redesigned flow, OkBuddy guides users from start to finish in a matter of minutes, always emphasizing quick wins and clarity. The user is never stuck wondering “what do I do next?” or “why do I have to fill this out?” – each screen has an obvious purpose, whether it’s uploading a file, viewing a score, editing a section, or considering an upgrade. By prioritizing fast value delivery over exhaustive data input, we keep users engaged from the get-go [OBJ] [OBJ]. By branching the experience based on the presence of a job description, we ensure the content of the advice always feels relevant.

By keeping the UI minimal and one-action-at-a-time, we reduce cognitive load and make the complex task of CV writing feel simple. And by integrating a fair freemium model, we create a sustainable business model without sacrificing user trust – free users get a great product, and those who need more can choose to pay for advanced help, with that option presented at just the right moments[OBJ].

The end result is a frictionless CV preparation flow that quickly gets users to their goal (a better resume and higher job match confidence), while also paving a path for OkBuddy’s monetization through genuine value-add features. This design balances user experience with business needs, creating a win-win scenario: users land interviews faster with improved CVs, and OkBuddy grows through satisfied users and premium conversions.